DOM Manipulations and events

-Pratiksha Dhavale

Javascript functions

A JavaScript function is a block of code designed to perform a particular task.

A JavaScript function is executed when "something" invokes it (calls it).

A JavaScript function is defined with the function keyword, followed by a name, followed by parentheses ().

Function names can contain letters, digits, underscores, and dollar signs (same rules as variables).

The parentheses may include parameter names separated by commas: (*parameter1, parameter2, ...*)

The code to be executed, by the function, is placed inside curly brackets: {}

Example

```
function myFunction(p1, p2) {
  return p1 * p2;
}
```

Parameters

```
Let mult = myFunction(5,2);        //Function invoking/function call
```

Function expression

The Javascript **Function Expression** is used to define a function inside any expression. The Function Expression allows us to create an anonymous function that doesn’t have any function name which is the main difference between Function Expression and Function Declaration.

Syntax:

let *variableName* = function*(x, y)* { *statements...* return *(z)* };

Arrow functions

Arrow functions were introduced in ES6.

Arrow functions allow us to write shorter function syntax:

```
let myFunction = (a, b) => a * b;
```

Call stack

A call stack is a mechanism for an interpreter (like the JavaScript interpreter in a web browser) to keep track of its place in a script that calls multiple functions — what function is currently being run and what functions are called from within that function, etc.

Example

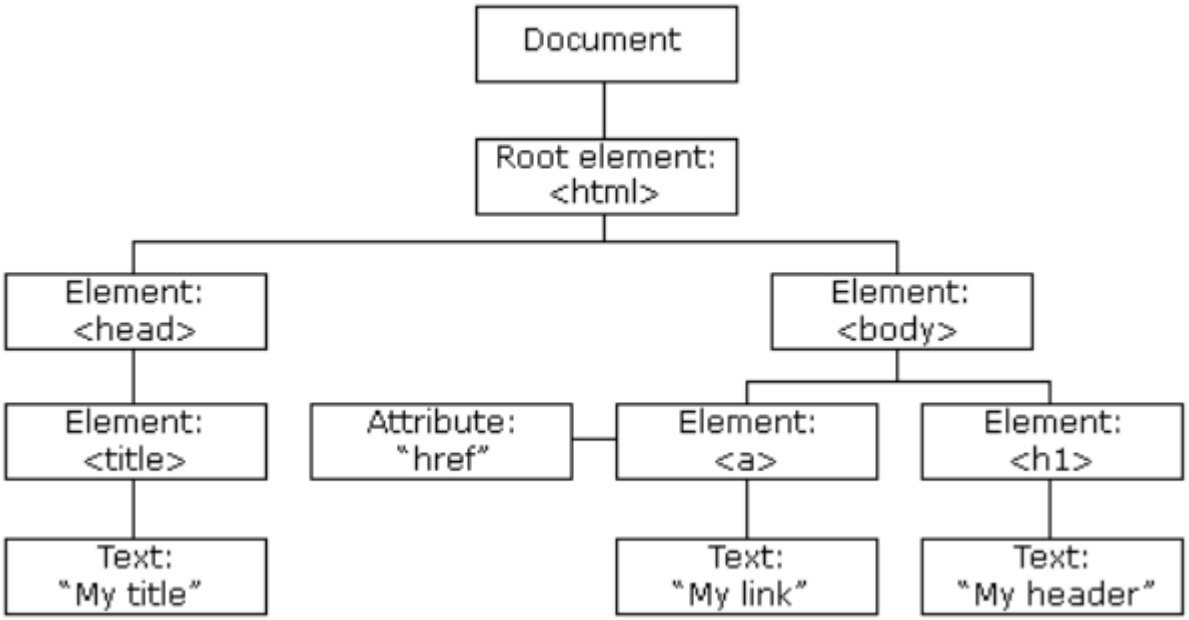
```
Function add(a,b){
Return a+b;
}
Function avg(a,b){
Return add(a,b)/2
}
Let result = avg(5,5)
```

Document Object Model

The Document Object Model (*DOM*) is the data representation of the objects that comprise the structure and content of a document on the web.

The Document Object Model (DOM) is a programming interface for web documents. It represents the page so that programs can change the document structure, style, and content. The DOM represents the document as nodes and objects; that way, programming languages can interact with the page.

Example


Document
Root element:
<html>
Element:
<head>
Element:
<body>
Element:
<title>
Attribute:
"href"
Element:
<a>
Element:
<h1>
Text:
"My title"
Text:
"My link"
Text:
"My header"

JS and DOM

With the object model, JavaScript gets all the power it needs to create dynamic HTML:

- JavaScript can change all the HTML elements in the page
- JavaScript can change all the HTML attributes in the page
- JavaScript can change all the CSS styles in the page
- JavaScript can remove existing HTML elements and attributes
- JavaScript can add new HTML elements and attributes
- JavaScript can react to all existing HTML events in the page
- JavaScript can create new HTML events in the page

Example

```
<html>
<body>

<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML = "Hello World!";
</script>

</body>
</html>
```

Finding HTML elements

Methods	Description
document.getElementById(*id*)	Find an element by element id
document.getElementsByTagName(*name*)	Find an element by tag name
document.getElementsByClassName(*name*)	Find an element by class name

Changing HTML elements

Property	Description
element.innerHTML = *new html content*	Change the inner HTML of an element
element.attribute = new value	Change the attribute value of an HTML element
element.style.*property = new style*	Change the style of an HTML element

Adding and Deleting elements

Method	Description
document.createElement(*element*)	Create an HTML element
document.removeChild(*element*)	Remove an HTML element
document.appendChild(*element*)	Add an HTML element
document.replaceChild(*new, old*)	Replace an HTML element
document.write(*text*)	Write into the HTML output stream

Jaavascript Events

HTML events are "things" that happen to HTML elements.

When JavaScript is used in HTML pages, JavaScript can "react" on these events.

some examples of HTML events:

- An HTML web page has finished loading
- An HTML input field was changed
- An HTML button was clicked

Some common events

Event	Description
onchange	An HTML element has been changed
onclick	The user clicks an HTML element
onmouseover	The user moves the mouse over an HTML element
onmouseout	The user moves the mouse away from an HTML element
onkeydown	The user pushes a keyboard key
onload	The browser has finished loading the page

Event listeners

The addEventListener() method attaches an event handler to the specified element.

The addEventListener() method attaches an event handler to an element without overwriting existing event handlers.

You can add many event handlers to one element.

You can add many event handlers of the same type to one element, i.e two "click" events.

You can add event listeners to any DOM object not only HTML elements. i.e the window object.

The addEventListener() method makes it easier to control how the event reacts to bubbling.

Example

```
document.getElementById("myBtn").addEventListener("click",
displayDate);
```

An event listener contains three parameters and it can be defined using the following syntax.

```
<element>.addEventListener(<eventName>,
        <callbackFunction>, {capture : boolean});
```

Event Bubbling

The process of propagating from the closest element to the farthest away element in the DOM (Document Object Modal) is called event bubbling.

Event Capturing

the process of propagating from the farthest element to the closest element in the DOM is called event capturing.